Hands-On Generative AI

Foundations of AI and Machine Learning

Day 1 - Session 1

KavinSchool

Welcome

- Introductions: your name, your team, and one thing you want to build with AI
- Course goals: go from ML basics to fine-tuning, RAG (retrieval-augmented generation), and agents in three days
- Everything hands-on runs in Google Colab, no local install required

> By the end of Day 1 you will have trained a classifier, an image model, a VAE (variational autoencoder), and generated images with diffusion.

Why Study Generative AI?

- **Adoption is now default:** teams ship features built on LLMs, and understanding what happens underneath separates people who can debug and extend these systems from people who only wire together demos
- **The stack moves fast:** embeddings, fine-tuning, RAG, and agents are the primitives behind most production AI work today, and hands-on practice is the fastest way to keep pace
- **Judgment matters more than API calls:** evaluating vendor claims, estimating cost, and choosing prompting versus RAG versus fine-tuning requires understanding tradeoffs, not just calling an endpoint

The Three-Day Roadmap

Day	Theme	You will build
Day 1	AI/ML and generative architectures	Classifier, image model, VAE, diffusion
Day 2	Natural language processing (NLP), pre-trained models, agents	Embeddings, BERT/GPT, prompting, an agent
Day 3	Fine-tuning, RAG, deployment	LoRA model, RAG pipeline, capstone

Environment Check

Before we start, confirm your setup:

- A Google account so you can open notebooks in Colab
- Open the Day 1 lab folder in the companion repo
- Run the first cell of Lab 1.1 to confirm the runtime starts

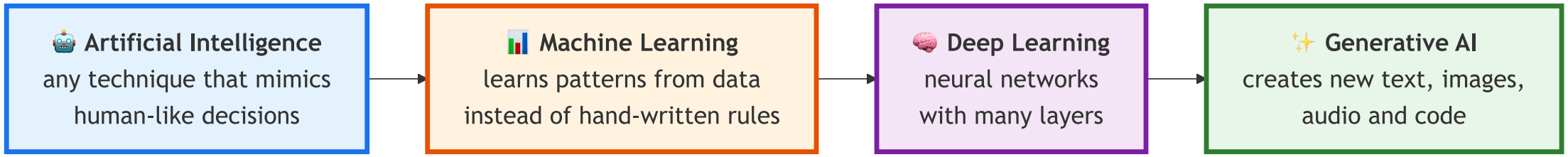
> No GPU needed today. Every Day 1 lab runs on a free Colab CPU runtime.

What Is Machine Learning?

Learning patterns from data instead of writing rules

KavinSchool

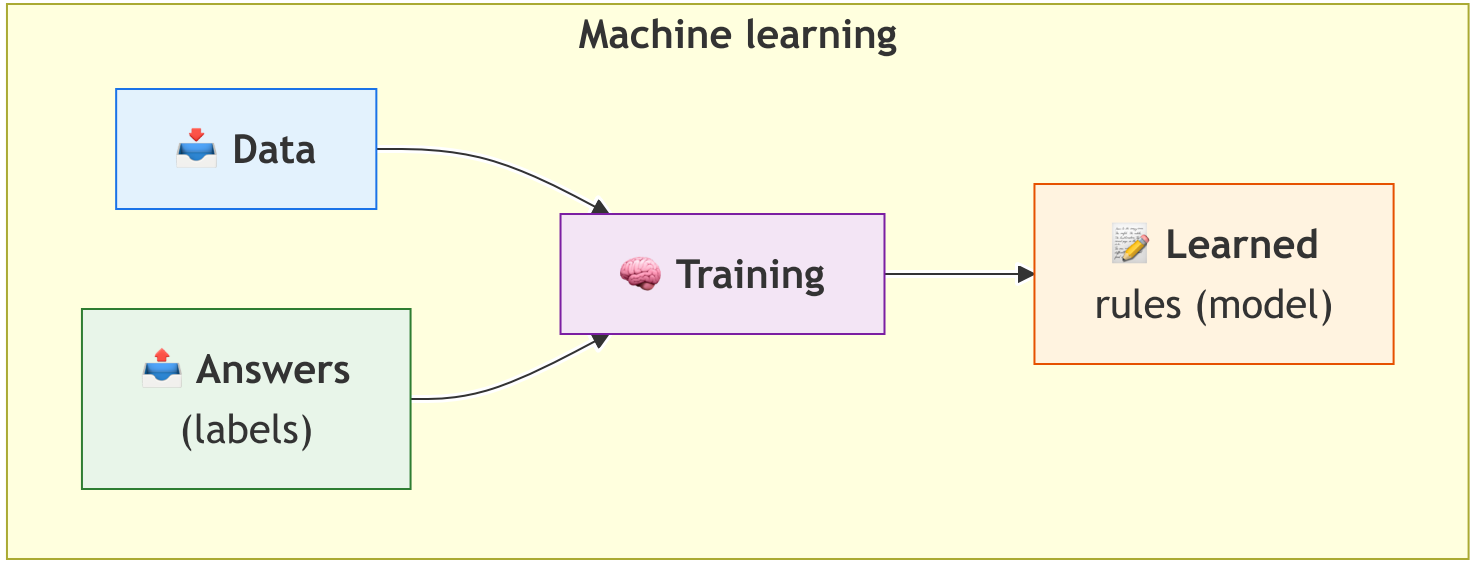
AI, ML, DL, and Generative AI

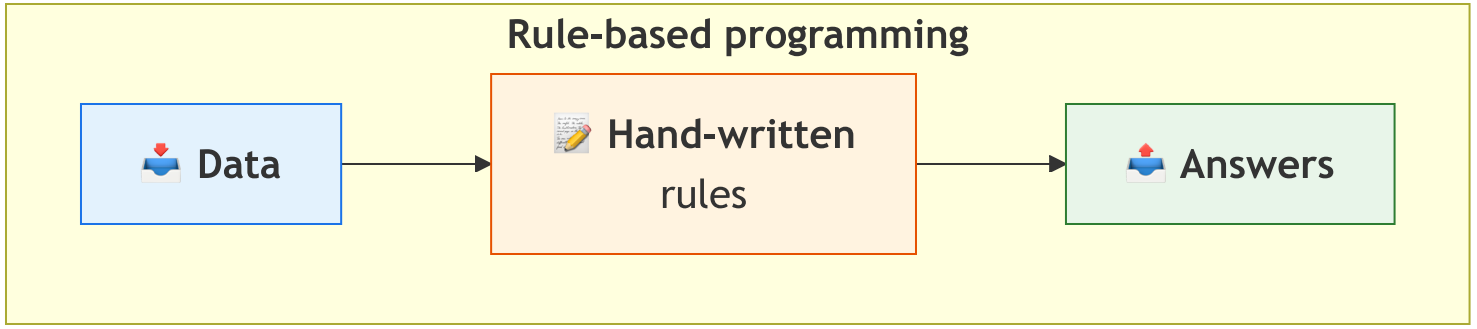


Rule-Based vs Machine Learning

Rule-based code needs a human to write every rule. Machine learning shows the model examples and lets it infer the rules.

Rule-Based vs Machine Learning





When to Reach for ML

Machine learning fits problems where:

- The rules are hard to write by hand (recognizing a cat, reading handwriting)
- Patterns shift over time (spam, fraud, demand)
- You have data that pairs inputs with the outcome you care about

Plain code is still better when the rules are simple, fixed, and easy to state.

What Is a Model?

A trained model is a mathematical function that:

- Takes in input data (features)
- Returns a prediction
- Bases that prediction on patterns learned during training

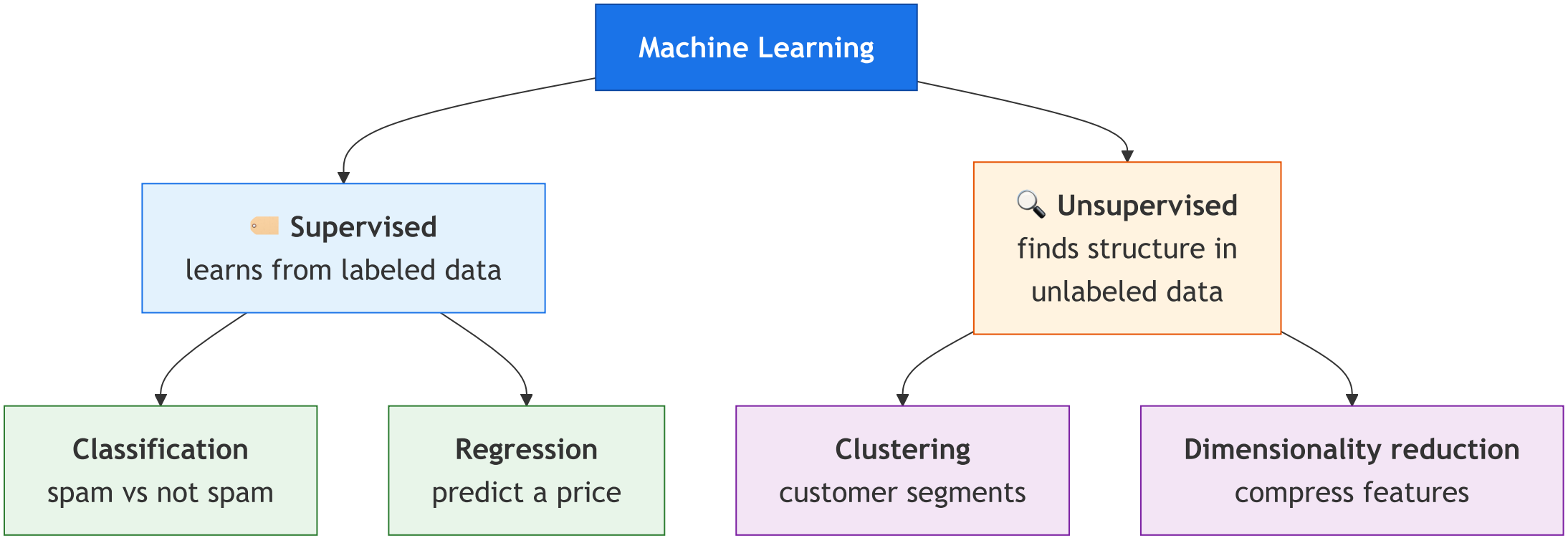
The training process is how those patterns get baked into the function.

Supervised and Unsupervised Learning

Two ways a model learns from data

KavinSchool

Two Families of Learning



Supervised Learning

The training data includes the answer (a label).

- **Classification:** predict a category (spam or not spam, digit 0-9)
- **Regression:** predict a number (house price, tomorrow's demand)

The model learns to map inputs to the known labels, then predicts labels for new inputs.

Unsupervised Learning

The training data has no labels. The model finds structure on its own.

- **Clustering:** group similar items (customer segments)
- **Dimensionality reduction:** compress many features into a few

Useful when labeling is expensive or you are exploring the data.

Where You See This Every Day

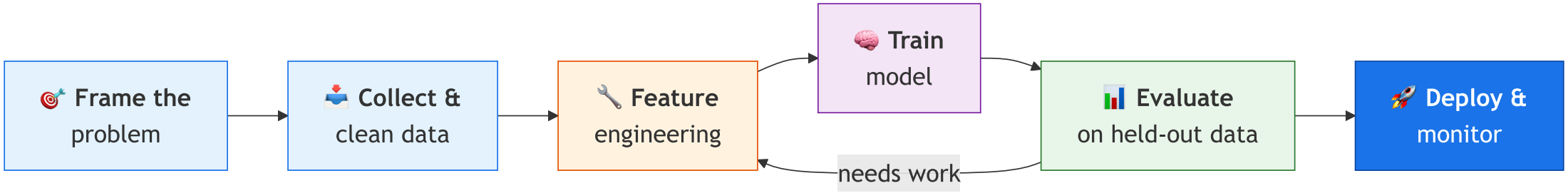
Task	Type	Example
Spam filter	Classification	Inbox vs junk
Price estimate	Regression	Home valuation
Recommendations	Clustering	"Customers also bought"
Fraud alerts	Anomaly detection	Unusual transactions

Building and Evaluating a Model

From raw data to a score you can trust

KavinSchool

The ML Workflow



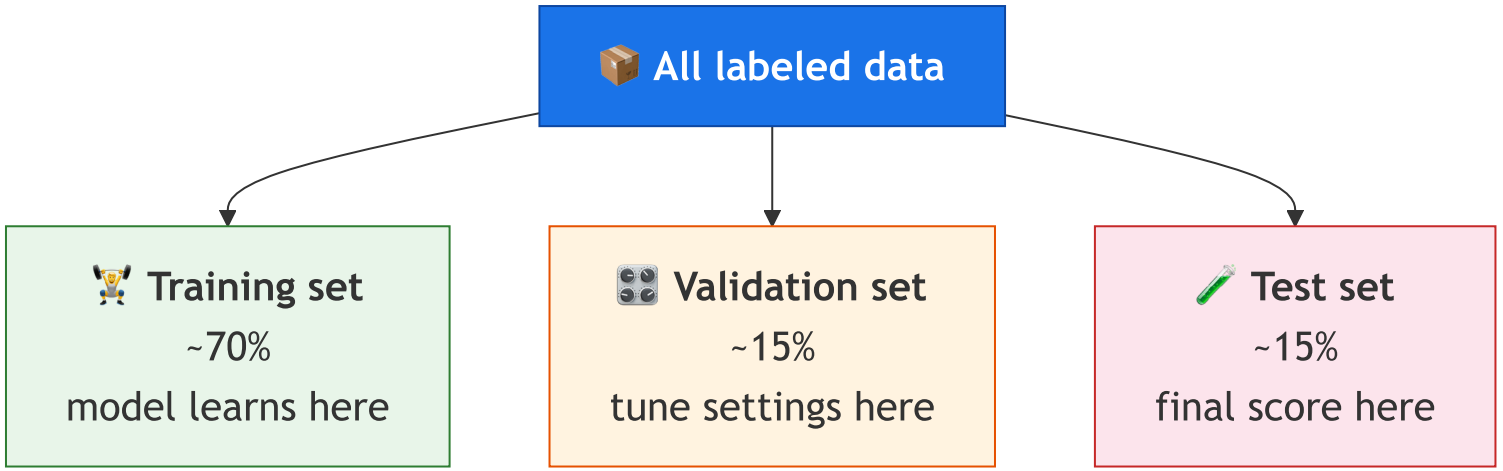
Most real projects loop back from evaluation to earlier steps several times.

Data Preprocessing and Features

- **Preprocessing:** clean missing values, scale numbers, encode categories
- **Feature engineering:** turn raw data into signals the model can use

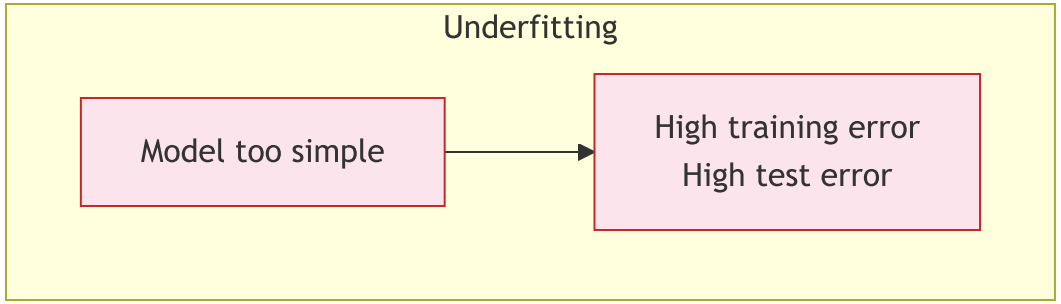
Good features often matter more than the choice of algorithm.

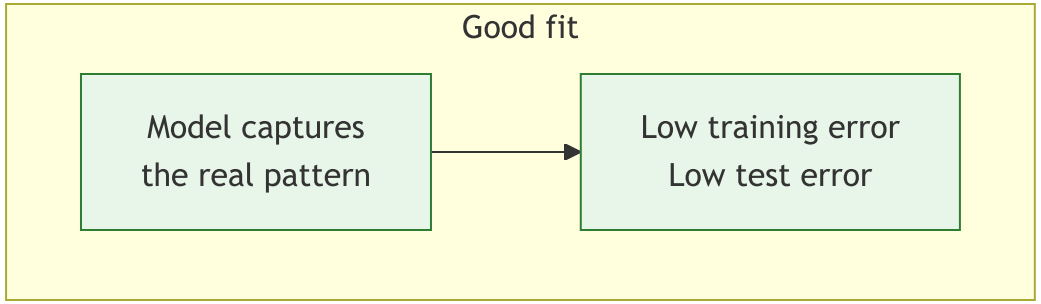
Train, Validate, Test

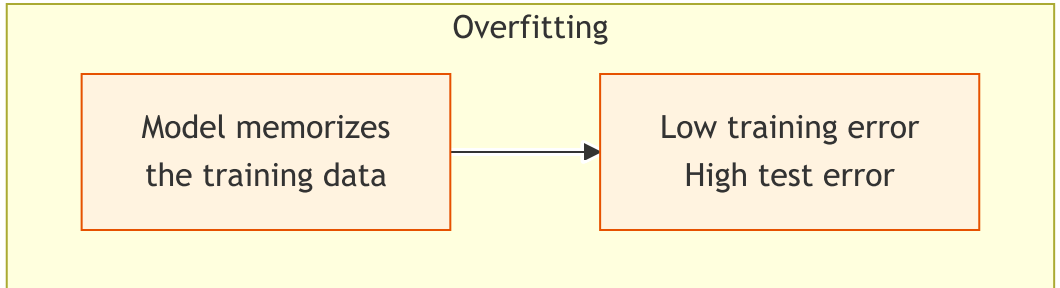


Never score your model on data it trained on. The test set is your honest estimate of real-world performance.

Overfitting and Underfitting





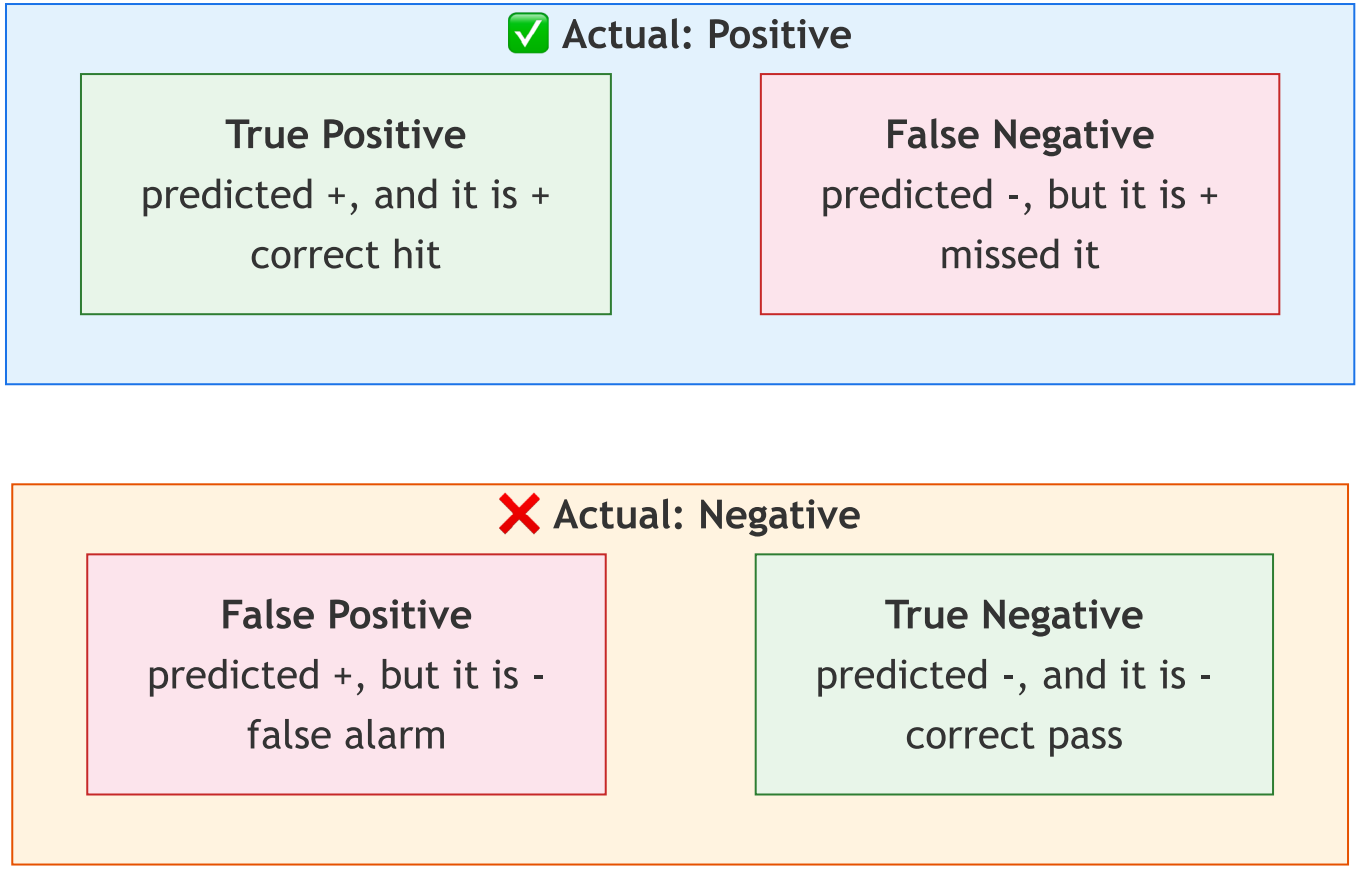


Spotting Overfitting

- **Underfit:** high error on both training and test data (model too simple)
- **Overfit:** low training error, high test error (model memorized noise)
- **Good fit:** low error on both, and it generalizes to new data

A large gap between training and test scores is the warning sign.

The Confusion Matrix



Precision, recall, and F1 are all ratios built from these four counts.

Evaluation Metrics

Metric	Answers	Watch out for
Accuracy	What fraction did we get right?	Misleading on imbalanced data
Precision	Of positives predicted, how many were right?	
Recall	Of actual positives, how many did we catch?	
F1	Balance of precision and recall	

Breakout Lab 1.1

Train and evaluate a classifier (optional)

KavinSchool

Lab 1.1: Train and Evaluate a Classifier

Open `day1/lab1_1_classifier/start.ipynb` in Colab.

- Load a small tabular dataset and split it into train and test sets
- Train a classifier with scikit-learn
- Report accuracy, precision, recall, and F1
- Compare train vs test scores to see overfitting in action

> Stretch goal: add regularization or try a second model and compare.

Key Takeaways

1. Machine learning learns rules from data instead of hand-coding them
2. Supervised learning uses labels; unsupervised learning finds structure
3. Good features and clean data drive results as much as the algorithm
4. Always evaluate on held-out data and watch the train-test gap
5. Pick metrics that match the problem, not accuracy alone

Questions?

Next: Deep Learning Primer

Morning break: 15 minutes

KavinSchool